Materiał 04 - Authentication, authorization i konfiguracja

Cel

Do tej pory każdy, kto znał adres API, mógł zrobić wszystko: zmienić status dowolnego zadania, dodać zadanie do dowolnego projektu. W tym materiale dodamy użytkowników i kontrolę dostępu.

Po jego przejściu powinieneś rozumieć:

- różnicę między **authentication** a **authorization** oraz między `401` a `403`,
- dlaczego hasła **hashujemy**, a nie szyfrujemy, i dlaczego nie używamy do tego SHA-256,
- jak działa **bcrypt**: salt i cost factor,
- czym jest **JWT**, z czego się składa i czego **nie** daje (np. nie szyfruje danych),
- jak napisać middleware `authenticate` i `authorize`,
- czym jest **ownership** i dlaczego jego sprawdzenie należy do service, a nie do middleware,
- dlaczego tożsamości użytkownika **nigdy** nie bierzemy z body requestu,
- jak zarządzać konfiguracją i sekretami (zasada **fail fast**),
- czym jest **OpenAPI** i jak udostępnić dokumentację API przez Swagger UI.

Kontynuujemy aplikację Task Manager z materiału 03.

> **Wersje użyte w materiale:** Node.js 22, `bcryptjs@3`, `jsonwebtoken@9`, `swagger-ui-express@5`, `zod@4`. Kod był uruchamiany i testowany na tych wersjach.

---

1. Authentication vs authorization

Dwa pojęcia, które łatwo pomylić:

	Authentication	**Authorization**
Pytanie	**Kim jesteś?**	**Co możesz zrobić?**

Przykład	login + hasło, token	"tylko ADMIN może archiwizować projekt"
Błąd HTTP	`401 Unauthorized`	`403 Forbidden`
Kolejność	najpierw	potem (wymaga znanej tożsamości)

Nazwa `401 Unauthorized` jest historycznie myląca - w praktyce oznacza "**nie wiem, kim jesteś**" (brak tokenu, zły token, token wygasł). `403 Forbidden` oznacza "**wiem, kim jesteś, ale nie wolno ci tego zrobić**".

Prosta reguła: jeżeli zalogowanie się (albo odświeżenie tokenu) mogłoby pomóc - `401`. Jeżeli nie pomoże - `403`.

---

2. Przechowywanie haseł

Nigdy plain text

Jeżeli baza wycieknie (a wyciekają regularnie - backup na publicznym serwerze, SQL injection, były pracownik), atakujący zna hasła wszystkich użytkowników. A ponieważ ludzie używają tych samych haseł w wielu miejscach - również do ich poczty i banku.

Hashowanie, nie szyfrowanie

- **Szyfrowanie** jest odwracalne - kto ma klucz, odczyta hasło. Klucz musi gdzieś leżeć na serwerze, więc wycieknie razem z bazą.
- **Hash** jest jednokierunkowy. Z hasha nie da się odtworzyć hasła. Przy logowaniu hashujemy podane hasło i porównujemy wyniki.

Dlaczego nie SHA-256?

SHA-256 jest **zaprojektowany, żeby być szybki**. Nowoczesna karta graficzna liczy miliardy hashy SHA-256 na sekundę. Atakujący z wyciekniętą bazą po prostu sprawdza wszystkie popularne hasła i ich warianty.

Do haseł używamy funkcji **celowo wolnych**: bcrypt, scrypt, Argon2. W tym materiale - **bcrypt**.

Jak działa bcrypt

```
$2b$12$Nq8Wv3o0hG7mJ8R0x1yZ2uQm3m9eW1a7oFv8kLr3Yd6b5Z0pQe1Hy
 |  |  |                     |
 |  |  salt (22 znaki)       hash (31 znaków)
 |  cost factor = 12
 wersja algorytmu
```

1. **Salt** - losowy ciąg generowany osobno dla **każdego** hasła i zapisywany razem z hashem. Dwóch użytkowników z hasłem `password123` ma **różne** hashe. Atakujący nie może raz policzyć tabeli "hasło → hash" i użyć jej dla całej bazy - musi łamać każde hasło osobno.
2. **Cost factor** - `12` oznacza 2^12 rund. Każde +1 podwaja czas liczenia. Hash z cost 12 liczy się dziesiątki-setki milisekund. Dla użytkownika przy logowaniu to niezauważalne, dla atakującego sprawdzającego miliardy haseł - zabójcze.

Cały ciąg (wersja + cost + salt + hash) zapisujemy w **jednej** kolumnie. Przy weryfikacji bcrypt odczytuje z niego salt i cost.

Instalacja

```
npm install bcryptjs
```

Używamy `bcryptjs` - implementacji w czystym JavaScript. Pakiet `bcrypt` jest szybszy, ale wymaga kompilacji kodu natywnego przy instalacji, co na Windowsie i w niektórych obrazach Docker sprawia problemy. API obu pakietów jest prawie identyczne.

```
const bcrypt = require("bcryptjs");

const hash = await bcrypt.hash("moje-tajne-haslo", 12);
const ok = await bcrypt.compare("moje-tajne-haslo", hash); // true
```

Ważne - limit 72 bajtów

bcrypt bierze pod uwagę tylko pierwsze **72 bajty** hasła. Dłuższe hasło zostanie po cichu obcięte - `"a".repeat(72) + "X"` i `"a".repeat(72) + "Y"` dadzą ten sam wynik `compare`. Dlatego w validation ustawiamy maksymalną długość hasła. (Precyzyjnie: limit dotyczy **bajtów** w UTF-8, a `z.string().max(72)` liczy znaki - polskie litery i emoji zajmują więcej niż 1 bajt. Dla projektu na zajęcia limit znaków w zupełności wystarcza.)

---

3. Tabela użytkowników

```
// src/db/migrations/20261027120000_create_users.js
exports.up = async function (knex) {
  await knex.schema.createTable("users", table => {
    table.increments("id").primary();
    table.string("email", 254).notNullable().unique();
    table.string("password_hash", 100).notNullable();
    table.string("role", 20).notNullable().defaultTo("USER");
    table.timestamp("created_at", { useTz:
true }).notNullable().defaultTo(knex.fn.now());
  });
  await knex.raw(`ALTER TABLE users ADD CONSTRAINT users_role_check CHECK
(role IN ('USER', 'ADMIN'))`);

  await knex.schema.alterTable("tasks", table => {

table.integer("created_by").references("id").inTable("users").onDelete("SET
NULL");
  });
};

exports.down = async function (knex) {
  await knex.schema.alterTable("tasks", table =>
table.dropColumn("created_by"));
  await knex.schema.dropTable("users");
};
```

Zwróć uwagę: to **nowa** migracja. Nie edytujemy migracji z materiału 03.

Kolumna `tasks.created_by` jest nullable, bo w bazie istnieją już zadania sprzed wprowadzenia użytkowników. Migracja dodająca kolumnę `NOT NULL` do tabeli z danymi wymagałaby najpierw uzupełnienia wartości (tzw. *backfill*).

Admina nie da się utworzyć przez rejestrację (zawsze dostaje rolę `USER`). Tworzymy go w seedzie:

```
// src/db/seeds/01_users.js
const bcrypt = require("bcryptjs");

exports.seed = async function (knex) {
  await knex.raw("TRUNCATE tasks, projects, users RESTART IDENTITY CASCADE");

  const passwordHash = await bcrypt.hash("admin-password-123", 12);
  await knex("users").insert({ email: "admin@example.com", password_hash:
passwordHash, role: "ADMIN" });
};
```

> Seedy uruchamiane są alfabetycznie. Seed z użytkownikami ma prefiks `01_`, a seed z projektami i zadaniami zmieniamy na `02_` - zadania odwołują się do użytkowników (`created_by`), więc użytkownicy muszą istnieć wcześniej. `TRUNCATE ... CASCADE` w pierwszym seedzie czyści też tabele zależne, dlatego czyszczenie robimy tylko raz, na początku.

---

4. Rejestracja

```
// src/modules/auth/auth.schemas.js
const { z } = require("zod");

const registerBody = z.object({
  email: z.email().toLowerCase(),
  password: z.string()
```

```
    .min(10, "Password must have at least 10 characters")
    .max(72, "Password can have at most 72 characters")
}).strict();

const loginBody = z.object({
  email: z.email().toLowerCase(),
  password: z.string().min(1)
}).strict();

module.exports = { registerBody, loginBody };
```

`.toLowerCase()` - `Jan@Example.com` i `jan@example.com` to w praktyce ten sam adres. Bez normalizacji można by zarejestrować dwa konta i nie dałoby się zalogować, wpisując adres inną wielkością liter.

`.strict()` - klient, który wyśle `{ "email": "...", "password": "...", "role": "ADMIN" }`, dostanie `400`. Bez `.strict()` pole `role` zostałoby co prawda zignorowane, ale jawny błąd jest lepszy - widać, że ktoś próbuje czegoś, czego API nie wspiera.

Długość zamiast "wielka litera + cyfra + znak specjalny" - aktualne zalecenia (np. NIST SP 800-63B) stawiają na długość hasła, a odradzają wymuszanie reguł złożoności, które prowadzą do haseł typu `Haslo123!`.

```
// src/modules/auth/auth.service.js (fragment)
async function register({ email, password }) {
  const existing = await usersRepository.findByEmail(email);
  if (existing) throw new ConflictError("Email already registered",
"EMAIL_TAKEN");

  const passwordHash = await bcrypt.hash(password, BCRYPT_ROUNDS);
  return usersRepository.create({ email, passwordHash, role: "USER" });
}
```

Dwie rzeczy:

1. **Rola jest ustawiana przez backend** (`role: "USER"`), nigdy na podstawie danych od klienta.
2. Sprawdzenie `findByEmail` to znowu **check-then-act** - dwa równoległe requesty z tym samym

emailem mogą oba przejść. Dlatego kolumna `email` ma `UNIQUE` w bazie, a error handler mapuje błąd `23505` na `409` (materiał 03). Sprawdzenie w service daje ładny komunikat, constraint daje gwarancję.

Odpowiedź rejestracji przechodzi przez mapper, który **nie zawiera** `passwordHash`:

```
const toUserResponse = u => ({ id: u.id, email: u.email, role: u.role,
createdAt: new Date(u.createdAt).toISOString() });
```

To jest moment, w którym DTO z materiału 02 przestaje być "teorią". Gdybyśmy zwrócili `res.json(user)`, hash hasła trafiłby do klienta.

---

5. Sesja czy token?

Po poprawnym logowaniu server musi jakoś "pamiętać", że kolejne requesty pochodzą od tego samego użytkownika. HTTP jest bezstanowy - każdy request jest niezależny.

	Sesja (session)	**Token (JWT)**
Gdzie jest stan	na serwerze (pamięć, Redis, baza)	w samym tokenie, u klienta
Co dostaje klient	losowe ID sesji (zwykle w cookie)	podpisany token
Weryfikacja	odczyt sesji z magazynu	sprawdzenie podpisu (bez bazy)
Wylogowanie	usuń sesję na serwerze - działa natychmiast	token jest ważny do exp
Wiele instancji / mikroserwisy	wspólny magazyn sesji	wystarczy znać klucz

Oba podejścia są poprawne. Na zajęciach używamy **JWT**, bo jest powszechny w REST API konsumowanym przez SPA (Angular, React) i aplikacje mobilne.

6. JWT - jak to działa

JWT (*JSON Web Token*, RFC 7519) to trzy części oddzielone kropkami:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoiVVNFUiIsImlhdCI6MTc5MTk1OTM5O
SwiZXhwIjoxNzkxOTYwMjk5LCJzdWIiOiIxIn0.kX3...
|_____________ header _____________| |_____________________________ payload
_____________________________| |signature|
```

Po zdekodowaniu (**base64url**):

```
// header
{ "alg": "HS256", "typ": "JWT" }

// payload
{ "role": "USER", "iat": 1791959399, "exp": 1791960299, "sub": "1" }
```

Signature = HMAC-SHA256(`header.payload`, **sekret**). Sekret zna tylko server.

Ważne - JWT nie jest zaszyfrowany

Payload to zwykły base64url - **każdy** może go odczytać (wklej token na jwt.io). Podpis gwarantuje tylko, że nikt go **nie zmienił**. Jeżeli użytkownik zmieni w payloadzie `"role": "USER"` na `"ADMIN"`, podpis przestanie pasować i server odrzuci token.

Wniosek: w payloadzie **nie umieszczamy** haseł, hashy, numerów PESEL ani niczego poufnego. Tylko to, co potrzebne do autoryzacji.

Standardowe pola (claims)

Claim	Znaczenie
`sub`	*subject* - kogo dotyczy token (ID użytkownika, jako **string**)
`iat`	*issued at* - kiedy wystawiono (sekundy od 1970-01-01)
`exp`	*expiration* - do kiedy ważny

Implementacja

```
npm install jsonwebtoken
```

```
// src/modules/auth/auth.service.js
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const { ConflictError, UnauthorizedError } = require("../../errors/AppError");

const BCRYPT_ROUNDS = 12;

function createAuthService({ usersRepository, jwtSecret, jwtExpiresIn }) {
  async function register({ email, password }) {
    const existing = await usersRepository.findByEmail(email);
    if (existing) throw new ConflictError("Email already registered",
"EMAIL_TAKEN");

    const passwordHash = await bcrypt.hash(password, BCRYPT_ROUNDS);
    return usersRepository.create({ email, passwordHash, role: "USER" });
  }

  async function login({ email, password }) {
    const user = await usersRepository.findByEmail(email);
    const passwordOk = user ? await bcrypt.compare(password,
user.passwordHash) : false;
    if (!user || !passwordOk) throw new UnauthorizedError("Invalid email or
password");

    const accessToken = jwt.sign(
      { role: user.role },
```

```
      jwtSecret,
      { subject: String(user.id), expiresIn: jwtExpiresIn, algorithm:
"HS256" }
    );
    return { accessToken, tokenType: "Bearer", expiresIn: jwtExpiresIn };
  }

  return { register, login };
}

module.exports = { createAuthService };
```

Jeden komunikat dla złego emaila i złego hasła. Gdyby API zwracało "user not found" vs "wrong password", atakujący mógłby sprawdzić, które adresy mają konto w systemie (*user enumeration*), a potem atakować tylko te.

> Uczciwie: rejestracja z `409 EMAIL_TAKEN` i tak zdradza, że email istnieje. W systemach o wysokich wymaganiach rejestracja zawsze odpowiada "sprawdź skrzynkę" i wysyła maila. Na potrzeby zajęć `409` jest akceptowalne - ważne, żebyś rozumiał kompromis.

`subject: String(user.id)` - specyfikacja JWT (RFC 7519) definiuje `sub` jako string. Biblioteka `jsonwebtoken` przyjmie też liczbę wpisaną ręcznie do payloadu, ale trzymamy się standardu - inne biblioteki (np. na frontendzie albo w innym serwisie) mogą być mniej tolerancyjne. Dlatego w middleware zamieniamy `sub` z powrotem na liczbę (`Number(payload.sub)`).

`expiresIn: "15m"` - krótki czas życia ogranicza szkody, jeżeli token wycieknie. Ponieważ JWT nie da się "unieważnić" przed `exp` bez dodatkowego mechanizmu, krótkie `exp` jest najprostszą obroną. W produkcji stosuje się dodatkowo **refresh token** (długo żyjący, przechowywany w bazie, można go odwołać), który służy do pobrania nowego access tokenu. To wykracza poza ten materiał.

Odpowiedź logowania:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "tokenType": "Bearer",
  "expiresIn": "15m"
}
```

7. Middleware `authenticate`

Klient wysyła token w nagłówku:

```
GET /api/tasks
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

```
// src/middleware/authenticate.js
const jwt = require("jsonwebtoken");
const { UnauthorizedError } = require("../errors/AppError");

function authenticate(jwtSecret) {
  return (req, res, next) => {
    const header = req.get("Authorization");
    if (!header || !header.startsWith("Bearer ")) {
      return next(new UnauthorizedError());
    }

    const token = header.slice("Bearer ".length);

    try {
      const payload = jwt.verify(token, jwtSecret, { algorithms: ["HS256"] });
      req.user = { id: Number(payload.sub), role: payload.role };
      next();
    } catch (err) {
      const message = err.name === "TokenExpiredError" ? "Token expired" :
"Invalid token";
      next(new UnauthorizedError(message));
    }
  };
}

module.exports = authenticate;
```

`jwt.verify` sprawdza podpis **i** `exp`. Rzuca:

- `TokenExpiredError` - token wygasł,
- `JsonWebTokenError` - zły podpis, uszkodzony token, zły algorytm.

`algorithms: ["HS256"]` - jawnie mówimy, jaki algorytm akceptujemy. Historycznie zdarzały się ataki, w których atakujący zmieniał `alg` w nagłówku tokenu (np. na `none`), a biblioteka ufała temu, co jest w tokenie. Whitelisting algorytmu zamyka tę drogę.

`req.user` - od tego momentu każdy kolejny middleware i controller wie, kto wysłał request. To jest **jedyne** wiarygodne źródło tożsamości.

Klasy błędów rozszerzamy o dwie nowe:

```
// src/errors/AppError.js (dopisujemy)
class UnauthorizedError extends AppError {
  constructor(message = "Authentication required") { super(message, 401,
"UNAUTHORIZED"); }
}
class ForbiddenError extends AppError {
  constructor(message = "Insufficient permissions") { super(message, 403,
"FORBIDDEN"); }
}
```

---

8. Authorization - role i ownership

Mamy dwa rodzaje reguł dostępu:

1. Role-based - "tylko ADMIN może archiwizować projekty". Decyzja zależy wyłącznie od roli użytkownika i endpointu. Nadaje się do middleware:

```
// src/middleware/authorize.js
```

```
const { ForbiddenError, UnauthorizedError } = require("../errors/AppError");

function authorize(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user) return next(new UnauthorizedError());
    if (!allowedRoles.includes(req.user.role)) return next(new
ForbiddenError());
    next();
  };
}

module.exports = authorize;
```

```
router.patch("/:id/archive", authorize("ADMIN"), validate({ params:
idParams }), controller.archive);
```

2. Ownership - "użytkownik może zmienić status **swojego** zadania, ADMIN - każdego". Tu decyzja zależy od **danych**: kto jest autorem tego konkretnego zadania. Middleware tego nie wie - musiałby pobrać zadanie z bazy, a to robi service. Dlatego sprawdzenie ownership umieszczamy w **service**:

```
// src/modules/tasks/tasks.service.js (fragment)
async function changeStatus(id, newStatus, currentUser) {
  return runInTransaction(async ({ projects, tasks }) => {
    const task = await tasks.findById(id);
    if (!task) throw new NotFoundError("Task");

    const isOwner = task.createdBy === currentUser.id;
    if (!isOwner && currentUser.role !== "ADMIN") {
      throw new ForbiddenError("You can only change status of your own
tasks");
    }

    // ... state machine, WIP limit (bez zmian)
  });
}
```

Service dostaje `currentUser` jako **zwykły parametr** (`{ id, role }`), a nie `req`. Nadal nie zna Expressa.

Controller przekazuje `req.user`:

```
async changeStatus(req, res) {
  const task = await tasksService.changeStatus(req.validated.params.id,
req.validated.body.status, req.user);
  res.json(toTaskResponse(task));
}
```

`403` czy `404`?

Użytkownik Anna próbuje zmienić zadanie Jana. Możemy zwrócić:

- `403 Forbidden` - "to zadanie istnieje, ale nie jest twoje",
- `404 Not Found` - "dla ciebie to zadanie nie istnieje".

`404` **ukrywa istnienie** zasobu. Ma to znaczenie, gdy samo istnienie jest informacją (np. `GET /api/invoices/1543` zdradza, ile faktur jest w systemie, a `GET /api/users/jan.kowalski` - czy Jan ma konto). Tak robi np. GitHub dla prywatnych repozytoriów.

`403` jest czytelniejsze przy debugowaniu i w sytuacjach, gdy użytkownik i tak wie, że zasób istnieje (widzi zadanie na wspólnej liście projektu).

Oba podejścia są poprawne - ważne, żeby wybór był **świadomy i spójny** w całym API. W Task Managerze zadania są widoczne dla wszystkich członków projektu, więc używamy `403`.

Nigdy nie ufamy tożsamości z body

```
POST /api/tasks
{ "title": "Hack", "projectId": 1, "createdBy": 7 }
```

Gdyby service brał `createdBy` z body, każdy mógłby tworzyć zadania "w imieniu" innych

użytkowników. Tożsamość pochodzi **wyłącznie** z tokenu:

```
// service
return tasks.create({ ...input, status: "TODO", createdBy: currentUser.id });
```

A schemat Zod z `.strict()` odrzuci pole `createdBy` w body z błędem `400`.

To samo dotyczy filtrowania: jeżeli USER ma widzieć tylko swoje dane, to filtr `createdBy = currentUser.id` dokłada **backend**. Parametr `?userId=` od klienta może co najwyżej **zawęzić** wyniki dla ADMINa, nigdy rozszerzyć uprawnień USERa.

---

9. Konfiguracja i sekrety

Problem

```
// NIE RÓB TEGO
const token = jwt.sign(payload, "my-secret");
```

Sekret w kodzie oznacza, że:

- każdy z dostępem do repozytorium może wygenerować token ADMINa,
- sekret jest w **historii git** na zawsze (nawet po usunięciu z kodu),
- development i produkcja mają ten sam sekret.

Zasada

Konfiguracja, która różni się między środowiskami (dane bazy, sekrety, porty), pochodzi ze **zmiennych środowiskowych** (to jedna z zasad tzw. *The Twelve-Factor App*).

Moduł konfiguracji z walidacją

Zamiast czytać `process.env.JWT_SECRET` w wielu miejscach, mamy **jeden** moduł, który:

1. wczytuje `.env` (jeżeli istnieje),
2. **waliduje** wszystkie zmienne,
3. przy błędzie **zatrzymuje aplikację** od razu (*fail fast*).

```
// src/config/index.js
const { z } = require("zod");

try {
  process.loadEnvFile();
} catch {
  // brak pliku .env - np. w Dockerze zmienne przychodzą z zewnątrz
}

const envSchema = z.object({
  NODE_ENV: z.enum(["development", "test",
"production"]).default("development"),
  PORT: z.coerce.number().int().default(3000),
  DATABASE_URL: z.url(),
  JWT_SECRET: z.string().min(32, "JWT_SECRET must have at least 32
characters"),
  JWT_EXPIRES_IN: z.string().default("15m")
});

const result = envSchema.safeParse(process.env);

if (!result.success) {
  console.error("Invalid environment configuration:");
  for (const issue of result.error.issues) {
    console.error(`  ${issue.path.join(".")}: ${issue.message}`);
  }
  process.exit(1);
}

module.exports = Object.freeze(result.data);
```

Dlaczego **fail fast**? Bez walidacji aplikacja z brakującym `JWT_SECRET` wystartuje normalnie i padnie

dopiero przy pierwszym logowaniu (`jsonwebtoken` rzuci wtedy `secretOrPrivateKey must have a value`, a użytkownik zobaczy `500`). Jeszcze gorzej z sekretem typu `"secret"` - `jsonwebtoken` go zaakceptuje bez ostrzeżenia, a tak krótki klucz HMAC da się złamać brute force'em i podrobić token ADMINa. Lepiej, żeby aplikacja w ogóle nie wystartowała, z jasnym komunikatem:

```
Invalid environment configuration:
  JWT_SECRET: JWT_SECRET must have at least 32 characters
```

`knexfile.js` też korzysta teraz z modułu konfiguracji:

```
// knexfile.js
const config = require("./src/config");

module.exports = {
  client: "pg",
  connection: config.DATABASE_URL,
  migrations: { directory: "./src/db/migrations" },
  seeds: { directory: "./src/db/seeds" }
};
```

`.env` i `.env.example`

```
# .env.example - JEST w repozytorium, opisuje potrzebne zmienne
NODE_ENV=development
PORT=3000
DATABASE_URL=postgres://postgres:postgres@localhost:5432/taskmanager
JWT_SECRET=replace-with-at-least-32-random-characters
JWT_EXPIRES_IN=15m
```

`.env` (prawdziwe wartości) - w `.gitignore`. `.env.example` - w repozytorium, żeby inna osoba wiedziała, co ustawić.

Losowy sekret wygenerujesz tak:

```
node -e
"console.log(require('node:crypto').randomBytes(48).toString('base64'))"
```

Ważne - sekret trafił do repozytorium

Usunięcie pliku w kolejnym commicie **nie wystarczy** - sekret nadal jest w historii git. Jedyne poprawne działanie: **wygeneruj nowy sekret** (*rotation*) i uznaj stary za skompromitowany. Na GitHubie publiczne repozytoria są automatycznie skanowane przez boty szukające kluczy - liczy się to w minutach, nie dniach.

---

10. Dokumentacja API - OpenAPI i Swagger UI

OpenAPI to standard opisu REST API w formacie JSON lub YAML: endpointy, parametry, schematy body, możliwe odpowiedzi, sposób uwierzytelnienia. **Swagger UI** to narzędzie, które z takiego opisu generuje interaktywną stronę - można z niej wysyłać requesty.

```
npm install swagger-ui-express
```

Fragment opisu (`src/docs/openapi.json`):

```
{
  "openapi": "3.0.3",
  "info": { "title": "Task Manager API", "version": "1.0.0" },
  "components": {
    "securitySchemes": {
      "bearerAuth": { "type": "http", "scheme": "bearer", "bearerFormat":
"JWT" }
    }
  },
  "paths": {
```

```
    "/api/tasks/{id}/status": {
      "patch": {
        "tags": ["Tasks"],
        "summary": "Change task status",
        "security": [{ "bearerAuth": [] }],
        "parameters": [{ "name": "id", "in": "path", "required": true,
"schema": { "type": "integer" } }],
        "requestBody": {
          "required": true,
          "content": { "application/json": { "schema": {
            "type": "object",
            "required": ["status"],
            "properties": { "status": { "type": "string", "enum": ["TODO",
"IN_PROGRESS", "DONE"] } }
          } } }
        },
        "responses": {
          "200": { "description": "Updated task" },
          "400": { "description": "Validation error" },
          "401": { "description": "Missing or invalid token" },
          "403": { "description": "Not the owner and not ADMIN" },
          "404": { "description": "Task not found" },
          "409": { "description": "Invalid transition or WIP limit exceeded" }
        }
      }
    }
  }
}
```

```
// src/app.js (fragment)
const swaggerUi = require("swagger-ui-express");
const openapiDocument = require("./docs/openapi.json");

app.use("/api/docs", swaggerUi.serve, swaggerUi.setup(openapiDocument));
```

Pod `http://localhost:3000/api/docs` pojawi się dokumentacja. `securitySchemes` dodaje przycisk **Authorize** - wklejasz token i Swagger UI dołącza go do kolejnych requestów.

Dobra dokumentacja opisuje **wszystkie** możliwe odpowiedzi, a nie tylko `200`. Frontend developer

musi wiedzieć, że `PATCH .../status` może zwrócić `409` i co wtedy pokazać użytkownikowi.

> Ręczne pisanie JSON-a jest najprostsze i najbardziej przejrzyste na start, ale łatwo o rozjazd dokumentacji z kodem. W większych projektach dokumentację generuje się z kodu: np. z komentarzy JSDoc (`swagger-jsdoc`) albo bezpośrednio ze schematów Zod. W Spring Boot i NestJS generowanie jest wbudowane / dostępne jako oficjalny moduł.

---

11. Składamy wszystko razem

```
// src/modules/auth/auth.routes.js
const { Router } = require("express");
const validate = require("../../middleware/validate");
const { NotFoundError } = require("../../errors/AppError");
const { registerBody, loginBody } = require("./auth.schemas");

const toUserResponse = u => ({ id: u.id, email: u.email, role: u.role,
createdAt: new Date(u.createdAt).toISOString() });

function createAuthRouter({ authService, usersRepository, authenticate }) {
  const router = Router();

  router.post("/register", validate({ body: registerBody }), async (req, res)
=> {
    const user = await authService.register(req.validated.body);
    res.status(201).json(toUserResponse(user));
  });

  router.post("/login", validate({ body: loginBody }), async (req, res) => {
    res.json(await authService.login(req.validated.body));
  });

  router.get("/me", authenticate, async (req, res) => {
    const user = await usersRepository.findById(req.user.id);
    if (!user) throw new NotFoundError("User");
    res.json(toUserResponse(user));
  });
```

```
  return router;
}

module.exports = { createAuthRouter };
```

(Dla zwięzłości handlery auth są zdefiniowane bezpośrednio w routerze. W większym module wydzielisz je do controllera jak w materiale 02.)

```
// src/app.js
const express = require("express");
const swaggerUi = require("swagger-ui-express");
const openapiDocument = require("./docs/openapi.json");
const requestLogger = require("./middleware/requestLogger");
const authenticate = require("./middleware/authenticate");
const notFound = require("./middleware/notFound");
const errorHandler = require("./middleware/errorHandler");
const { createTasksService } = require("./modules/tasks/tasks.service");
const { createTasksController } = require("./modules/tasks/tasks.controller");
const { createTasksRouter } = require("./modules/tasks/tasks.routes");
const { createAuthService } = require("./modules/auth/auth.service");
const { createAuthRouter } = require("./modules/auth/auth.routes");

function createApp({ config, runInTransaction, usersRepository }) {
  const app = express();
  app.use(requestLogger);
  app.use(express.json());

  const requireAuth = authenticate(config.JWT_SECRET);

  const authService = createAuthService({
    usersRepository,
    jwtSecret: config.JWT_SECRET,
    jwtExpiresIn: config.JWT_EXPIRES_IN
  });
  const tasksService = createTasksService({ runInTransaction });

  app.use("/api/docs", swaggerUi.serve, swaggerUi.setup(openapiDocument));
  app.use("/api/auth", createAuthRouter({ authService, usersRepository,
authenticate: requireAuth }));
  app.use("/api/tasks", requireAuth,
```

```
createTasksRouter(createTasksController(tasksService)));

  app.use(notFound);
  app.use(errorHandler);
  return app;
}

module.exports = { createApp };
```

`app.use("/api/tasks", requireAuth, router)` - middleware `requireAuth` działa dla **wszystkich** ścieżek zadań. Nie da się zapomnieć o zabezpieczeniu nowego endpointu w tym module. `/api/auth/register` i `/api/auth/login` są publiczne (muszą być - inaczej nie dałoby się zalogować), a `/api/auth/me` ma `authenticate` dodany pojedynczo.

```
// src/server.js
const config = require("./config");
const db = require("./db/knex");
const { createApp } = require("./app");
const { createKnexUnitOfWork } = require("./modules/tasks/
tasks.repository.knex");
const { createUsersRepository } = require("./modules/auth/
users.repository.knex");

const app = createApp({
  config,
  runInTransaction: createKnexUnitOfWork(db).runInTransaction,
  usersRepository: createUsersRepository(db)
});

app.listen(config.PORT, () => console.log(`Server running on http://localhost:
${config.PORT}`));
```

Przepływ requestu z uwierzytelnieniem

```
PATCH /api/tasks/5/status         Authorization: Bearer eyJ...
  |
  v
```

```
requestLogger
  |
  v
express.json()
  |
  v
authenticate      -> brak / zły / wygasły token?  -> 401
  |                   req.user = { id: 2, role: "USER" }
  v
validate          -> złe dane?                    -> 400
  |
  v
controller        -> tasksService.changeStatus(5, "IN_PROGRESS", req.user)
  |
  v
service           -> brak zadania?                -> 404
  |                   nie właściciel i nie ADMIN?   -> 403
  |                   zła zmiana statusu / WIP?     -> 409
  v
repository -> PostgreSQL
```

Kolejność sprawdzeń nie jest przypadkowa: `401` przed `400` (nie walidujemy danych od kogoś, kogo nie znamy), `404` przed `403` (żeby wiedzieć, czyje jest zadanie, trzeba je najpierw znaleźć).

---

12. Przygotowanie pod frontend - CORS

Gdy frontend (np. Angular na `http://localhost:4200`) wyśle request do API na `http://localhost:3000`, przeglądarka go **zablokuje**. To nie błąd API - to mechanizm bezpieczeństwa przeglądarki (**Same-Origin Policy**): strona z jednego *origin* (schemat + host + port) nie może domyślnie czytać odpowiedzi z innego.

API musi jawnie zezwolić na konkretne originy nagłówkiem `Access-Control-Allow-Origin`. Najprościej pakietem `cors`:

```
npm install cors
```

```
```

```
const cors = require("cors");
app.use(cors({ origin: ["http://localhost:4200"] }));
```

Nie używaj `origin: "*"` "żeby działało" - pozwala to dowolnej stronie w internecie wywoływać Twoje API z przeglądarki użytkownika. Listę dozwolonych originów najlepiej też trzymać w konfiguracji (zmienna środowiskowa).

Postman i `curl` nie podlegają CORS - dlatego problem pojawia się dopiero przy frontendzie.

---

13. Podsumowanie

```
POST /api/auth/register  -> walidacja, bcrypt.hash, role = USER, odpowiedź bez
hasha
POST /api/auth/login     -> bcrypt.compare, jwt.sign({ role }, secret,
{ subject, expiresIn })
każdy chroniony request  -> Authorization: Bearer <token>
                         -> authenticate: jwt.verify -> req.user
                         -> authorize(...roles)      -> 403 dla złej roli
                         -> service: ownership       -> 403 / 404
```

Najważniejsze rzeczy:

- `401` = nie wiem, kim jesteś. `403` = wiem, ale nie wolno ci.
- Hasła **hashujemy** wolną funkcją z solą (bcrypt), nigdy SHA-256, nigdy plain text.
- JWT jest **podpisany, nie zaszyfrowany**. Payload może przeczytać każdy.
- Tożsamość użytkownika pochodzi **tylko** z zweryfikowanego tokenu, nigdy z body ani query.
- Role sprawdzamy w middleware. **Ownership** sprawdzamy w service, bo wymaga danych.
- Konfiguracja ze zmiennych środowiskowych, walidowana przy starcie (**fail fast**).
- Sekret, który trafił do repozytorium, jest spalony - trzeba go zmienić.
- Dokumentacja API opisuje **wszystkie** odpowiedzi, łącznie z błędami i sposobem

uwierzytelnienia.

W następnym materiale zajmiemy się testami automatycznymi, uruchamianiem całości przez Docker Compose, zdarzeniami (eventami) oraz CI w GitHub Actions.